

❄️ Snowflake COPY INTO – Real-Time Production Scenario (End-to-End) ❄️

Below is a **REAL-TIME PRODUCTION SCENARIO** explained **step by step**, using the **same COPY INTO concepts and commands you learned**, but shown **how it actually runs in a live company project**.

I'll explain this like **what happens daily in production, who does what, why each step exists, and what decisions are taken**.

🏢 Business Scenario (Example)

Company: Banking / Finance / E-commerce

Source System: Core Banking / ERP / CRM

Data Type: Employee / Customer / Transaction data

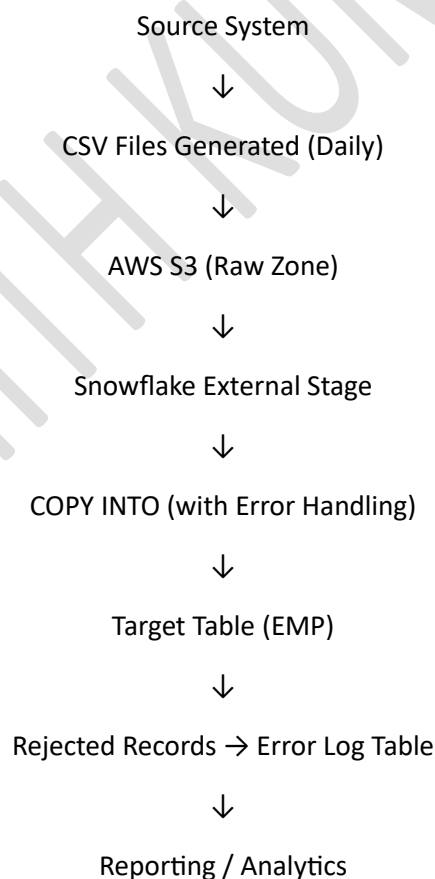
Frequency: Daily (Night Batch – 1 AM)

File Type: CSV

Cloud Storage: AWS S3

Target: Snowflake Enterprise Data Warehouse

🔄 Daily Real-Time Data Flow (High Level)



Roles Involved (Real Life)

Role	Responsibility
Source Team	Generate CSV files
Cloud Team	Maintain S3 bucket
Snowflake Dev	Write COPY logic
DBA / Data Engineer	Monitor loads
QA	Validate data
Business	Use reports

DAILY PRODUCTION PROCESS (STEP-BY-STEP)

◆ STEP 1: File Generation (Outside Snowflake)

What happens

- Source system exports **daily data**
- File naming standard followed

 Example:

emp_2025_01_19.csv

Why naming matters

- Used in **PATTERN**
- Helps in audit & reruns

◆ STEP 2: File Lands in S3 (Raw Zone)

s3://skillscaleraw/DW/employee/emp_2025_01_19.csv

Production Rules

- Files are **append-only**
 - No overwrite
 - IAM access restricted
-

◆ STEP 3: Snowflake Objects (Already Created)

These are **ONE-TIME setup** (not daily):

- Database
- Schema
- File Format
- External Stage
- Target Table

✓ In production, **we do NOT recreate these daily**

◆ STEP 4: Pre-Load Validation (Production Best Practice)

Command

```
LIST @my_external_stg/employee/;
```

Why in production

- Confirms file arrival
- Prevents empty loads
- Used in monitoring dashboards

If file missing

🚨 Alert raised

🚫 COPY job NOT triggered

◆ STEP 5: Optional File Preview (DEV / QA)

```
SELECT $1,$2,$3,$4
```

```
FROM @my_external_stg/employee/emp_2025_01_19.csv
```

```
(file_format=>'my_csv_format')
```

```
LIMIT 10;
```

Why

- Schema validation
- Detect column shift early

📌 Usually skipped in PROD automation

◆ STEP 6: COPY INTO with Production-Safe Settings

✓ REAL PRODUCTION COPY COMMAND

```
COPY INTO emp
FROM @my_external_stg/employee/
FILE_FORMAT = my_csv_format
PATTERN = '*.emp_2025_01_19.*.csv'
ON_ERROR = CONTINUE;
```

🧠 What happens internally (Production)

1. Snowflake scans only **today's file**
2. Applies CSV parsing rules
3. Loads **valid rows**
4. Rejects **bad rows**
5. Logs metadata in COPY_HISTORY

📊 Output (Production Result)

Metric	Value
Rows Loaded	9,950
Rows Rejected	50
Status	PARTIALLY_LOADED

✓ Business data is available

✗ Bad data isolated

◆ STEP 7: Capture Rejected Records (MANDATORY in PROD)

Why this is critical

- Auditing
- Data quality checks
- Regulatory compliance

Command

```
CREATE OR REPLACE TABLE emp_err_log AS
```

```
SELECT *
```

```
FROM TABLE(
```

```
  VALIDATE(emp, job_id=> '_last')
```

```
);
```

Result

- Stores:
 - Failed row data
 - Column name
 - Error reason
 - File name
-

◆ STEP 8: Notify Source Team (Operational Flow)

✉ Automated email / ServiceNow ticket:

“50 records failed in emp_2025_01_19.csv due to invalid salary data.”

✚ Source team fixes data next day

◆ STEP 9: Post-Load Audit Check

Command

```
SELECT *  
  
FROM TABLE(  
  
  INFORMATION_SCHEMA.COPY_HISTORY(  
    table_name=>'EMP',  
    start_time=>DATEADD(hour,-24,CURRENT_TIMESTAMP())  
  ));
```

Why

- Confirms file loaded
 - Tracks duplicates
 - SLA monitoring
-

◆ STEP 10: Prevent Duplicate Loads

Default Behavior

FORCE = FALSE

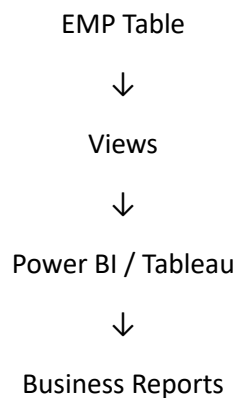
What Snowflake does

- Tracks file checksum
- Prevents re-loading same file

✦ If reprocessing required → FORCE = TRUE (manual approval)

◆ STEP 11: Downstream Consumption

Once data is loaded:



✓ Executives see updated data by morning

◆ STEP 12: Failure Scenarios & Decisions (REAL LIFE)

✗ Case 1: 100% File Corrupt

ON_ERROR = SKIP_FILE

- ✓ File skipped
 - 🚨 Incident raised
-

✗ Case 2: Data Type Errors

- ✓ Load valid rows
 - ✓ Capture error rows
 - ✓ No business impact
-

✗ Case 3: File Not Arrived

- 🚫 COPY not executed
 - 📧 Alert triggered
-

◆ STEP 13: Automation (Production Reality)

COPY command runs via:

- TASK
 - Stored Procedure
 - Airflow / ADF / Control-M
- ✦ Manual execution only in DEV
-

◆ STEP 14: Retention & Recovery

- Time Travel → accidental deletes
 - Fail-Safe → disaster recovery
 - COPY_HISTORY → audit (14 days)
 - ACCOUNT_USAGE → audit (365 days)
-

WHY THIS DESIGN IS USED IN PRODUCTION

Requirement	Solution
High volume	COPY INTO
Bad data	ON_ERROR
Audit	COPY_HISTORY
Compliance	Error tables
Performance	Bulk load
Automation	Tasks

INTERVIEW-LEVEL SUMMARY (YOU CAN SAY)

“In production, we receive daily CSV files in S3. Snowflake uses external stages and COPY INTO with ON_ERROR=CONTINUE to load valid data while rejecting bad rows. We capture rejected records using the VALIDATE function, audit loads using COPY_HISTORY, and prevent duplicate loads using Snowflake’s file tracking mechanism. The entire process is automated using tasks or orchestration tools.”